Interaktiv und sicher

Termin 6 · 17.11.2026

FÄLLIG
bis Termin 7 am 24.11.

ZEITBEDARF
ca. 3 Stunden

BAUSTEINE IM REPO
t6-dom-und-events, t6-zustand-und-render, t6-fehlersuche

KONTAKT
dhbw@theaisoftwarecompany.com

Ab heute reagiert eure Seite. Das ist der Termin, an dem aus einer Webseite eine Web-App wird – und der, an dem die meisten Fehler entstehen. Gut so: Fehlersuche ist diese Woche eine eigene Aufgabe.

Dazu kommt ein Sicherheits-Check. Der Oktober hat gezeigt, wie schnell vergessene Basics zum Problem werden – bei großen Anbietern genauso wie bei der Hochschul-IT. Euer Projekt geht in zwei Wochen live in die Bewertung, also prüft ihr es jetzt.

Teil A · Pflicht

A1 JavaScript einbinden

`app.js` anlegen und im `head` einbinden:

```
<script src="app.js" defer></script>
```

Wer mit Vite arbeitet, hat schon `<script type="module" src="/src/main.js">` – das verhält sich wie `defer`.

→ **Fertig, wenn** ein `console.log` beim Laden in der Konsole erscheint und `document.querySelector` ein Element findet – nicht `null`.

A2 Ein Formular ohne Reload

Fangt euer Formular aus Blatt 2 mit JavaScript ab:

- `submit`-Event, `event.preventDefault()`
- Eingabe auslesen, prüfen, verarbeiten
- Ergebnis in den DOM schreiben – mit `textContent`, nicht `innerHTML`

→ **Fertig, wenn** beim Absenden **nichts neu lädt** und die Adresszeile unverändert bleibt.

A3 Ein Zustand, eine Zeichenfunktion

Legt alle Daten, die eure Anzeige bestimmen, in **ein** Objekt. Schreibt **eine** Funktion, die daraus die Anzeige baut. Alles, was sich ändert, geht durch diese eine Funktion.

Mindestens **zwei** Dinge hängen an demselben Zustand – zum Beispiel eine Liste und ein Zähler, oder eine Liste und ein Filter.

→ **Fertig, wenn** Liste und Zähler nach jeder Aktion zusammenpassen, auch nach dem Löschen.

A4 Fünf Fehler finden

Der Baustein `t6-fehlersuche` enthält fünf eingebaute Fehler. Findet sie ohne KI-Hilfe – zumindest die ersten zehn Minuten.

Schreibt für jeden Fehler drei Zeilen: **Symptom → Ursache → Fix.**

→ **Fertig, wenn** die Seite tut, was sie soll, und eure fünf Notizen im Repo liegen (z. B. `notizen/fehlersuche.md`).

A5 KI-Log anfangen

Legt in der README den Abschnitt „KI-Log" an und füllt ihn mit den ersten echten Einträgen dieser Woche: welche Werkzeuge, ein Fall, in dem es geholfen hat, ein Fall, in dem die KI falsch lag.

→ **Fertig, wenn** mindestens ein Eintrag beschreibt, wo die KI danebenlag – und woran ihr es gemerkt habt.

A6 Sicherheits-Check

Geht die Checkliste aus Termin 6 für euer Projekt durch und haltet das Ergebnis in der README im Abschnitt „Sicherheit“ fest – pro Punkt eine Zeile, was ihr geprüft habt und wie:

1. **Keine Geheimnisse:** kein geheimer Key in Code, Git-Historie oder `VITE_`-Variablen; `.env` steht in `.gitignore`
2. **Fremde Daten als Text:** alles von Nutzern oder APIs per `textContent`, nicht per `innerHTML`
3. **Eingaben prüfen:** leer, zu lang, falscher Typ – mit einer verständlichen Meldung
4. **HTTPS live:** die Live-URL und alle Aufrufe laufen über `https://`
5. **GitHub gehärtet:** 2FA im Account, Dependabot und Secret Scanning im Repo aktiviert
6. **Saubere Fehler:** Meldungen für Nutzer ohne Stacktraces oder interne Details

Testet Punkt 2 ganz praktisch: Tippt `<img src=x onerror=alert(1)>` in jedes Eingabefeld eurer Seite.

→ **Fertig, wenn** der Abschnitt „Sicherheit“ in der README steht, Dependabot im Repo aktiv ist und beim Test aus Punkt 2 **kein** Fenster aufgeht.

Teil B · Bonus (+1 Punkt)

Merkt euch etwas über den Reload hinweg: `localStorage` beim Ändern schreiben, beim Start lesen.

Kleine Warnung als Teil der Aufgabe: Was passiert, wenn dort etwas steht, das nicht mehr zu eurem Code passt? Fangt den Fall ab.

Teil C · Verständnisfragen

1. Was ist der Unterschied zwischen `textContent` und `innerHTML` – und warum ist der zweite bei fremden Daten gefährlich?
2. Was macht `event.preventDefault()` bei einem `submit`, und was passiert ohne?
3. Warum steht `defer` am Script-Tag? Was passiert ohne, wenn das Script im `head` steht?

4. Warum ist eine Prüfung im Frontend („nur Admins sehen den Löschen-Knopf“) keine Sicherheitsmaßnahme? Wo gehört die Prüfung hin?
5. Was haben XSS und SQL-Injection gemeinsam – und wie verhindert man beides?
6. Nennt zwei der Vorfälle aus dem Oktober 2026 und je eine Maßnahme, die davor geschützt hätte.

So arbeitest du hier mit KI

Diese Woche ist die Woche, in der man am meisten lernen – oder am meisten verpassen – kann. JavaScript ist die Sprache, die Modelle am besten schreiben. Genau deshalb müsst ihr sie am besten lesen können.

Die Regel für dieses Blatt: **zehn Minuten selbst, dann fragen.** Und dann so:

> *„Hier ist mein Code und diese Fehlermeldung. Sag mir nicht die Lösung. Erklär mir, was die Meldung bedeutet und in welcher Zeile ich suchen soll."*

Und wenn ihr etwas übernommen habt:

> *„Erklär mir Zeile für Zeile, was dieser Code tut. Stell mir danach drei Fragen dazu, damit ich merke, ob ich es verstanden habe."*

Wer Aufgabe A4 an die KI weiterreicht, verschenkt genau die Fähigkeit, die im Job zählt: einen Fehler finden, den noch niemand beschrieben hat.

Für A6 dürft ihr die KI ausdrücklich als Prüfer einsetzen – mit dem Prompt aus den Folien („Geh die OWASP Top 10:2025 einzeln durch …“). Aber: Jeden Fund selbst nachvollziehen. Ein KI-Security-Check erfindet auch Probleme und übersieht andere.